

Sistemas Operacionais

Sistema de Arquivos - Parte 02

Professor: **Jeremias Moreira Gomes**

E-mail: jeremias.gomes@idp.edu.br

Introdução

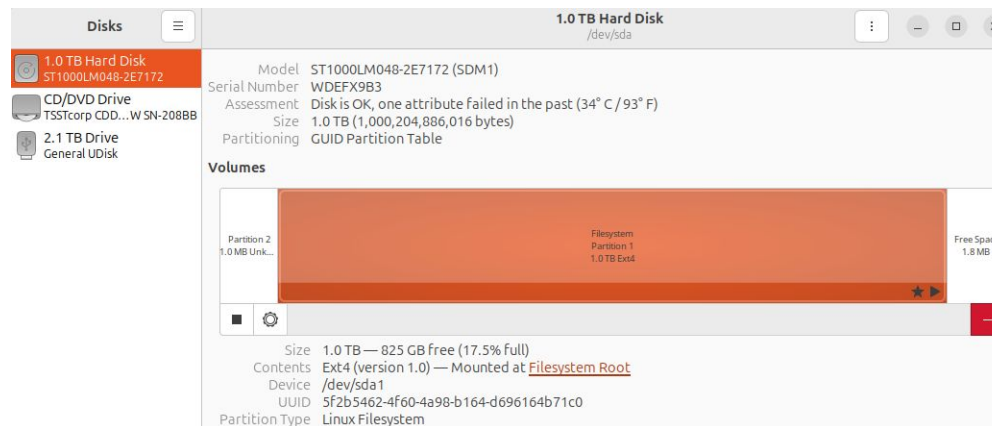
Implementação de Sistemas de Arquivos

Implementação de Sistemas de Arquivos

- Usuários estão preocupados com nomes, operações permitidas ou como é a árvore de diretórios
- Desenvolvedores precisam saber como arquivos são armazenados ou como o espaço de disco é gerenciado de maneira eficiente e confiável

Esquema de Sistema de Arquivos

- Sistemas de arquivos são armazenados em discos
- Discos pode ser dividida em uma ou mais partições
 - Cada partição possui seu próprio sistema de arquivos



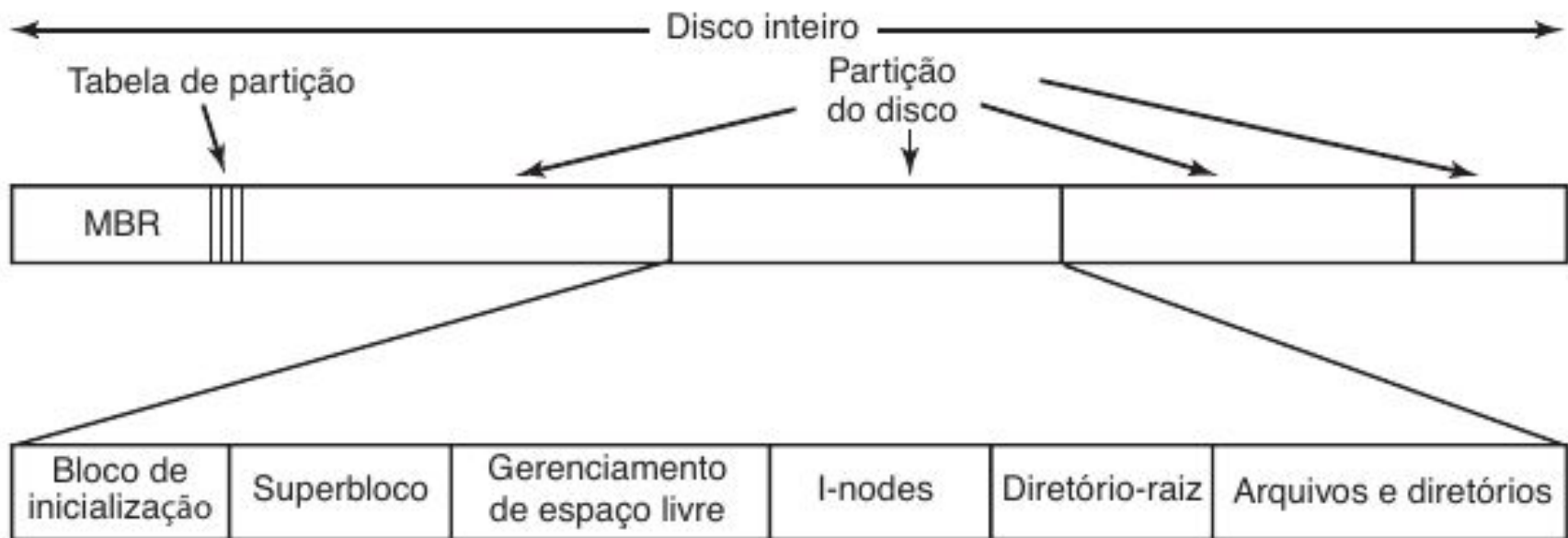
Esquema de Sistema de Arquivos

- O setor 0 é chamado de **MBR** (Master Boot Record)
 - **Inicialização do computador**
 - Todo disco reserva esse espaço
 - O fim da MBR contém a tabela de partição
 - Endereços de início e fim de cada partição
 - Uma dessas partições pode ser marcada como ativa*

Esquema de Sistema de Arquivos

- A partição ativa contém diversos setores
 - **Bloco de Inicialização**
 - De onde se carrega o SO
 - Toda partição possui, mesmo não possuindo um SO
 - **Superbloco**
 - Parâmetros chave a respeito do sistema de arquivos

Esquema de Sistema de Arquivos



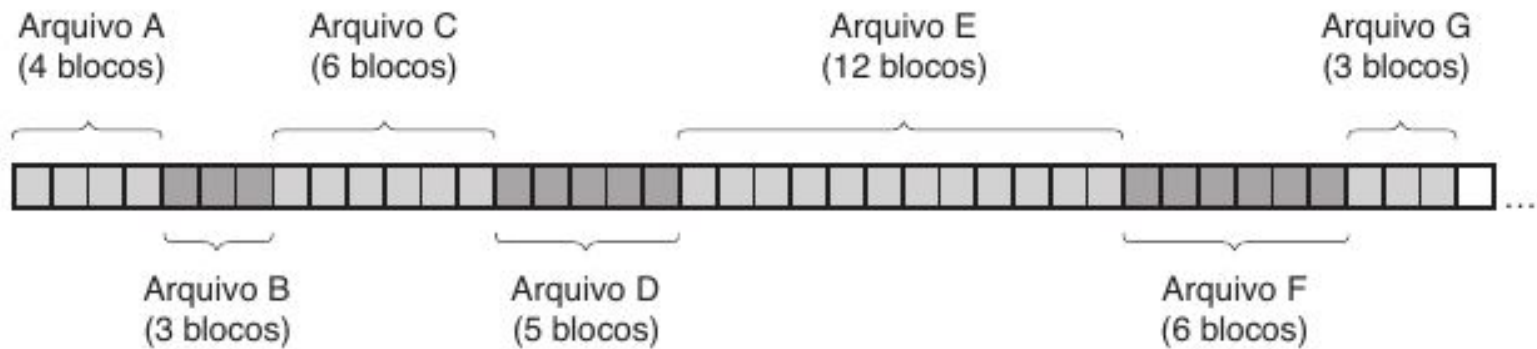
Implementação de Arquivos

- Existem vários métodos diferentes ao implementar sistemas de arquivos, em diferentes Sistemas Operacionais
 - Alocação Contígua
 - Alocação por Lista Encadeada
 - Alocação por Lista Encadeada com tabela em memória
 - I-nodes

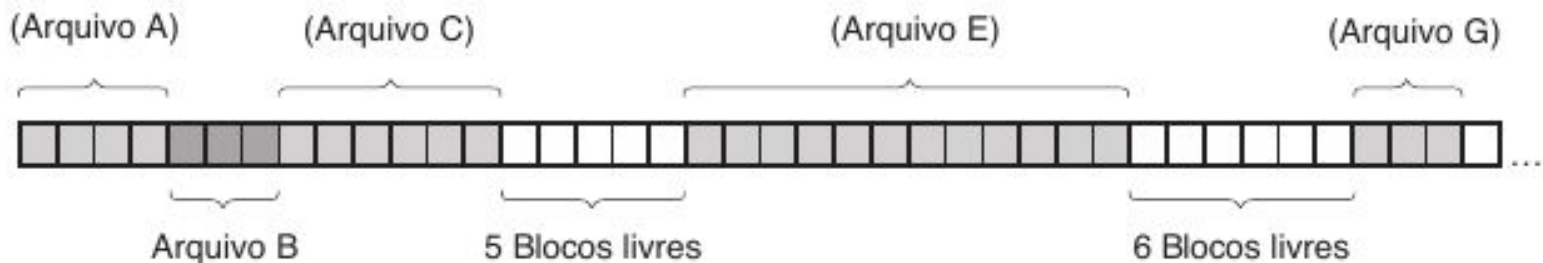
Implementação - Alocação Contígua

- Cada arquivo é uma execução contígua de blocos no disco
 - Exemplo:
 - Blocos de 1K, um arquivo de 50K ocupa 50 blocos
 - Arquivos sempre começam no início do bloco
 - Simples e performático
 - Tem problema de fragmentação

Implementação - Alocação Contígua



(a)



(b)

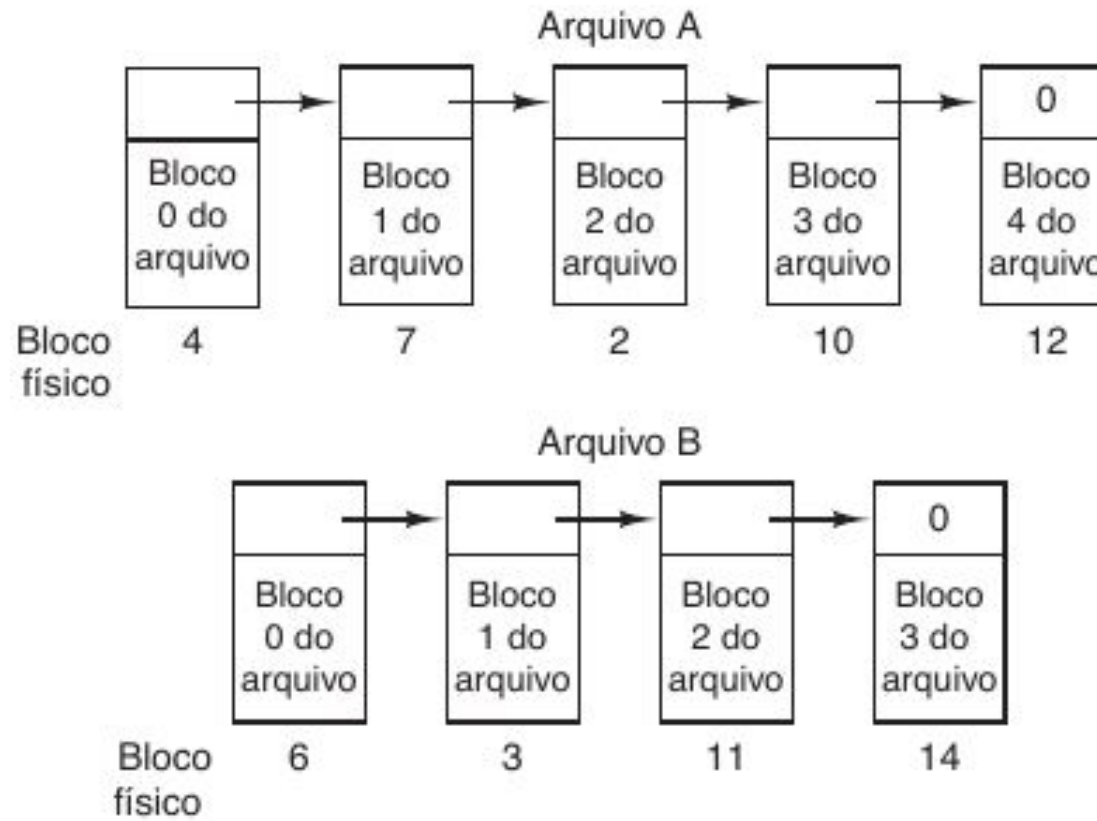
Implementação - Alocação Contígua

- CDs - Caso onde alocação contígua é OK (obsoleto né)
- DVD
 - O protocolo UDF (Universal Disk Format) utiliza apenas 30 bits para tamanho
 - Limita os arquivos a terem no máximo 1GB
- Blu Rays não tem esse problema (usam 64 bits) (UDF 2.6)

Implementação - Alocação por Lista Encadeada

- A primeira palavra de cada bloco é utilizada para armazenar um ponteiro
 - O restante do bloco para dados
- Sem espaços perdidos para fragmentação
- Leitura é direta, mas acesso aleatório é lento
- Há bytes ocupados para ponteiros agora

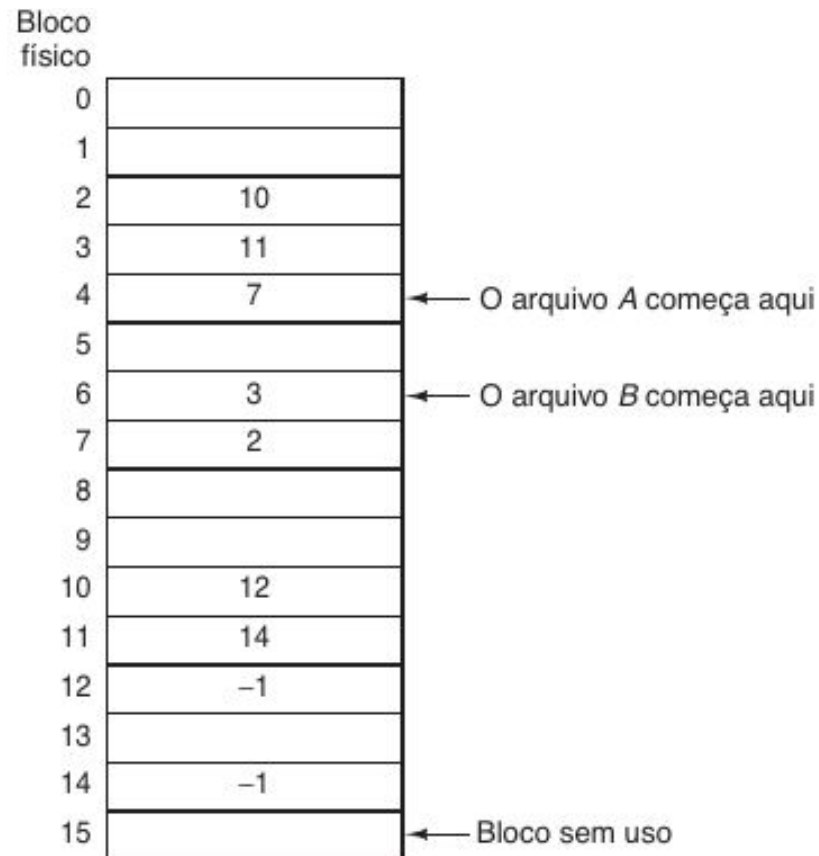
Implementação - Alocação por Lista Encadeada



Lista Encadeada com Tabela na Memória

- A ideia principal é colocar as palavras dos ponteiros de cada bloco em uma tabela na memória principal
 - No caso anterior, perdia-se uma vantagem do aproveitamento de potências de 2 para armazenamento
 - Agora a alocação inteira do bloco é utilizada para dados
- A tabela na RAM tem o nome de **File Allocation Table** (FAT)

Lista Encadeada com Tabela na Memória



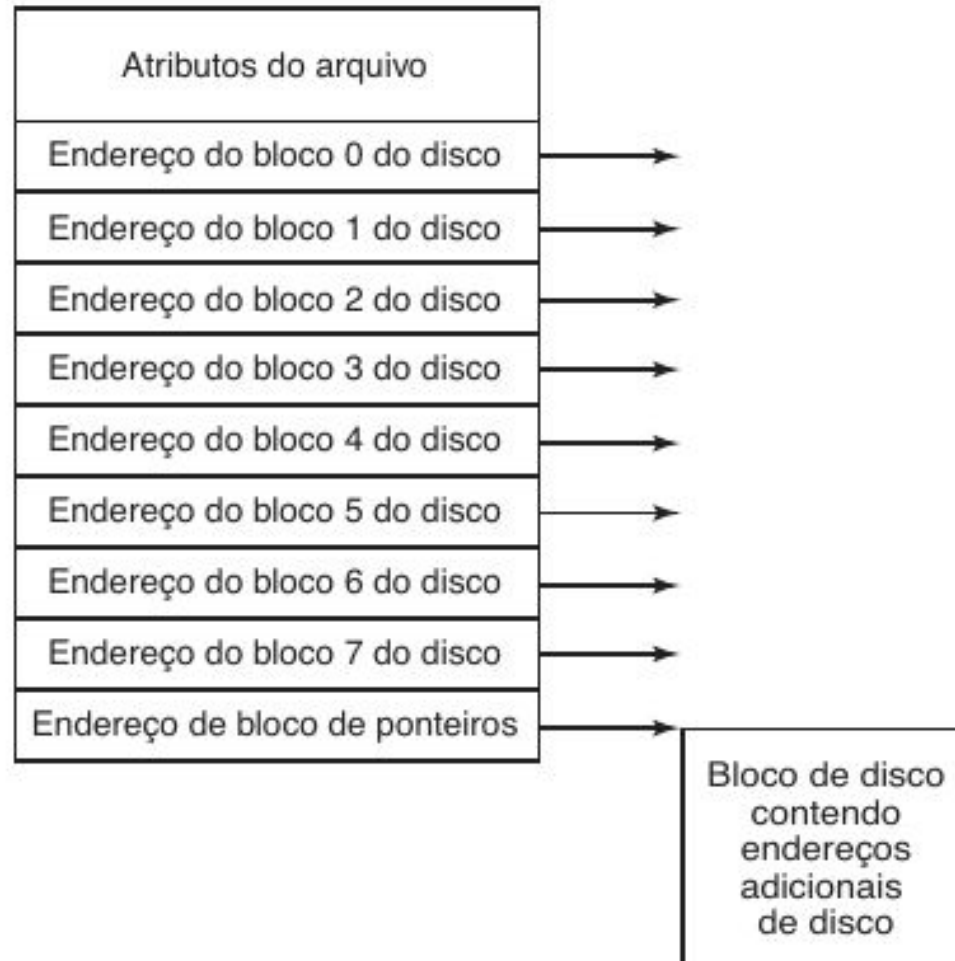
Lista Encadeada com Tabela na Memória

- Em sistemas FAT, a tabela precisa estar sempre em memória
 - Se um disco tem 1TB e o bloco é de 1K, utilizando entradas de 4 bytes, é necessário 3GB só para a tabela
 - Pouco prático para discos muito grandes

I-nodes

- Aqui a ideia base é realizar a associação de arquivos a uma estrutura de dados chamada i-node
 - Possui atributos e endereços dos blocos no disco
- I-nodes só vão para a memória se o arquivo estiver “aberto”
- Cada arquivo irá reservar um i-node
 - Onde estão as informações a respeito do arquivo

I-nodes

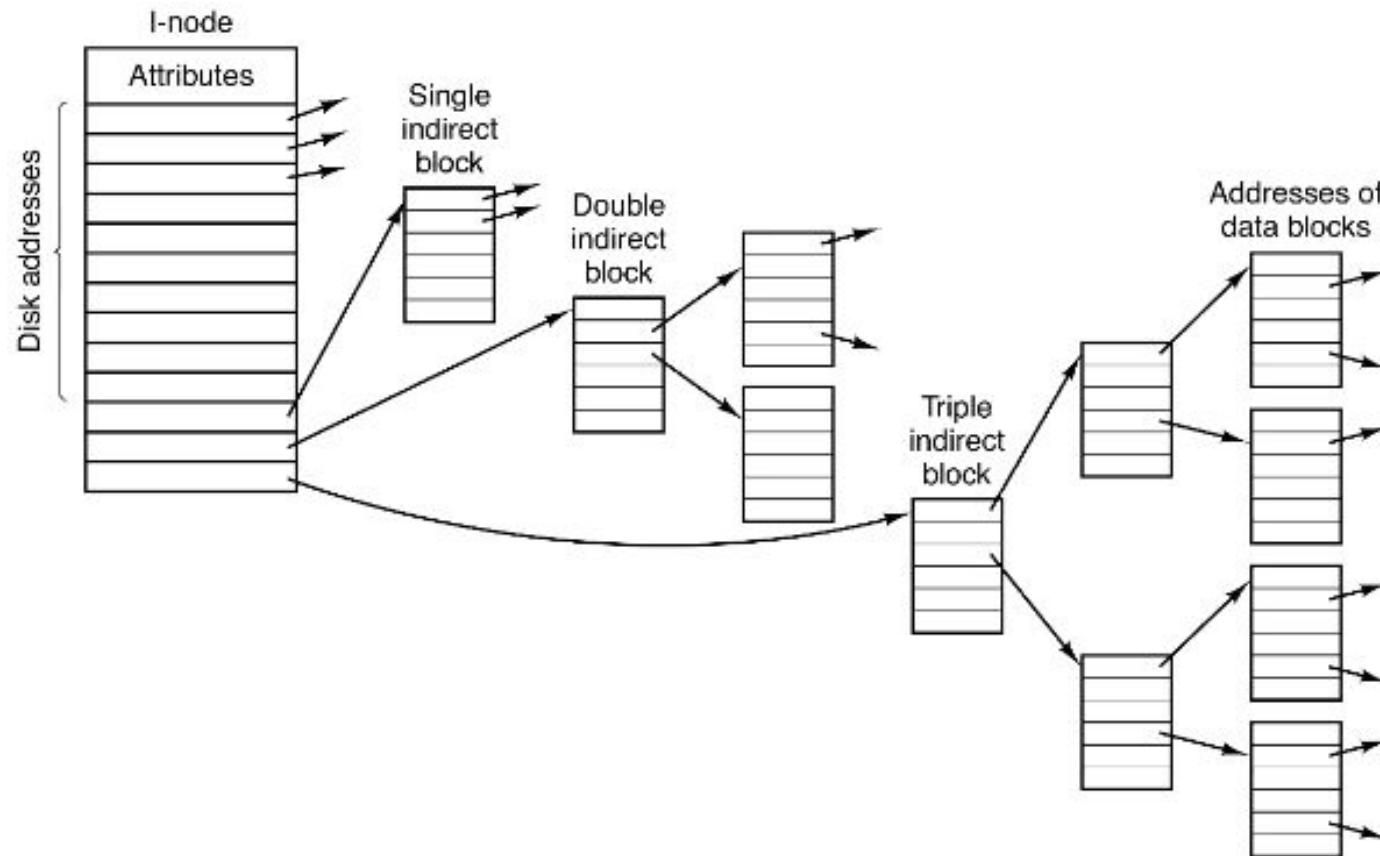


I-nodes - Estrutura Típica

Campo	Descrição	Unix
Número	Número do inode	inum
Dono	Número identificador do dono do arquivo	uid
Grupo	Número identificador do grupo do arquivo	gid
Tipo	Os tipos principais são arquivo e diretório	- ou d
Permissões	A lista de permissões, armazenada no formato rwx	----- (ugo)
Última modificação	Data/hora da última modificação	
Criação	Data/hora da criação do arquivo	
Tamanho	Tamanho do Arquivo	
Número de links	Número de (hard) links para esse arquivo	
Lista de Ponteiros	Ponteiros diretos e indiretos*	

*10 diretos (acesso rápido aos dados), 1 indireto simples (expande o número de blocos), 1 indireto duplo (suporta arquivos maiores) e 1 indireto triplo (suporta arquivos muito grandes)

I-nodes



I-nodes

- O sistema NTFS do Windows utiliza algo próximo
 - Com ajuste para arquivos muito grandes
- O sistema ext4, mais utilizado em SOs Linux, utiliza essa estrutura

```
struct ext4_dir_entry {  
    unsigned long inode; /* Inode number */  
    unsigned short rec_len; /* Directory entry length */  
    unsigned short name_len; /* Name length */  
    char name[up to 255]; /* File name */  
};
```

Implementação de Diretórios

- Antes de um arquivo ser lido, ele precisa ser aberto
 - SO usa nome+caminho para localizar o arquivo no disco
 - Nome pode indicar, dependendo do sistema de arquivos, endereço físico (alocação contígua), número do primeiro bloco (listas encadeadas) ou número do i-node
- Mapear nome do arquivo para localizar os dados

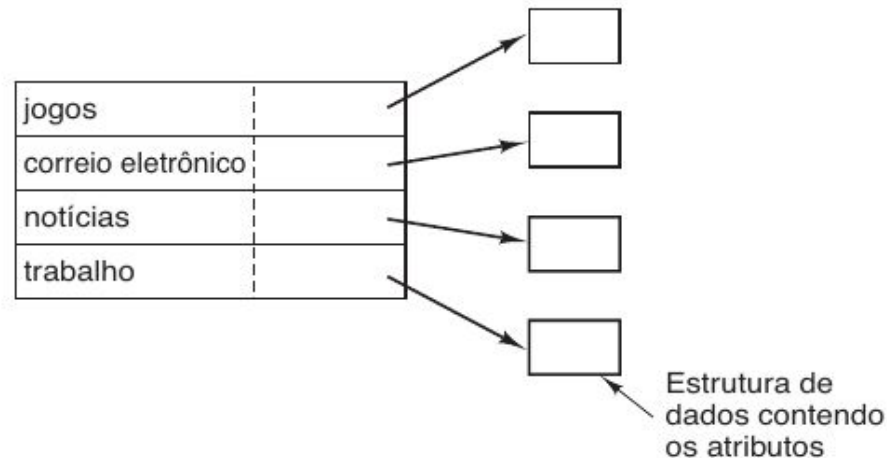
Implementação de Diretórios

- E onde os **atributos** devem ser armazenados? (1/2)
 - Uma opção é diretamente na entrada do diretório
 - Uma lista de entradas de tamanho fixo

jogos	atributos
correio eletrônico	atributos
notícias	atributos
trabalho	atributos

Implementação de Diretórios

- E onde os **atributos** devem ser armazenados? (2/2)
 - Sistemas que utilizam i-nodes podem armazenar nos próprios i-nodes



Implementação de Diretórios

- Um outro detalhe da implementação, diz respeito ao tamanho dos nomes dos arquivos (1/2)
 - Da maneira como atributos foram apresentados (slide anterior), um tamanho fixo é requerido
 - MSDOS com sua limitação de 8 + 3 caracteres
 - 255 chars p/ outros sistemas (desperdício de espaço)

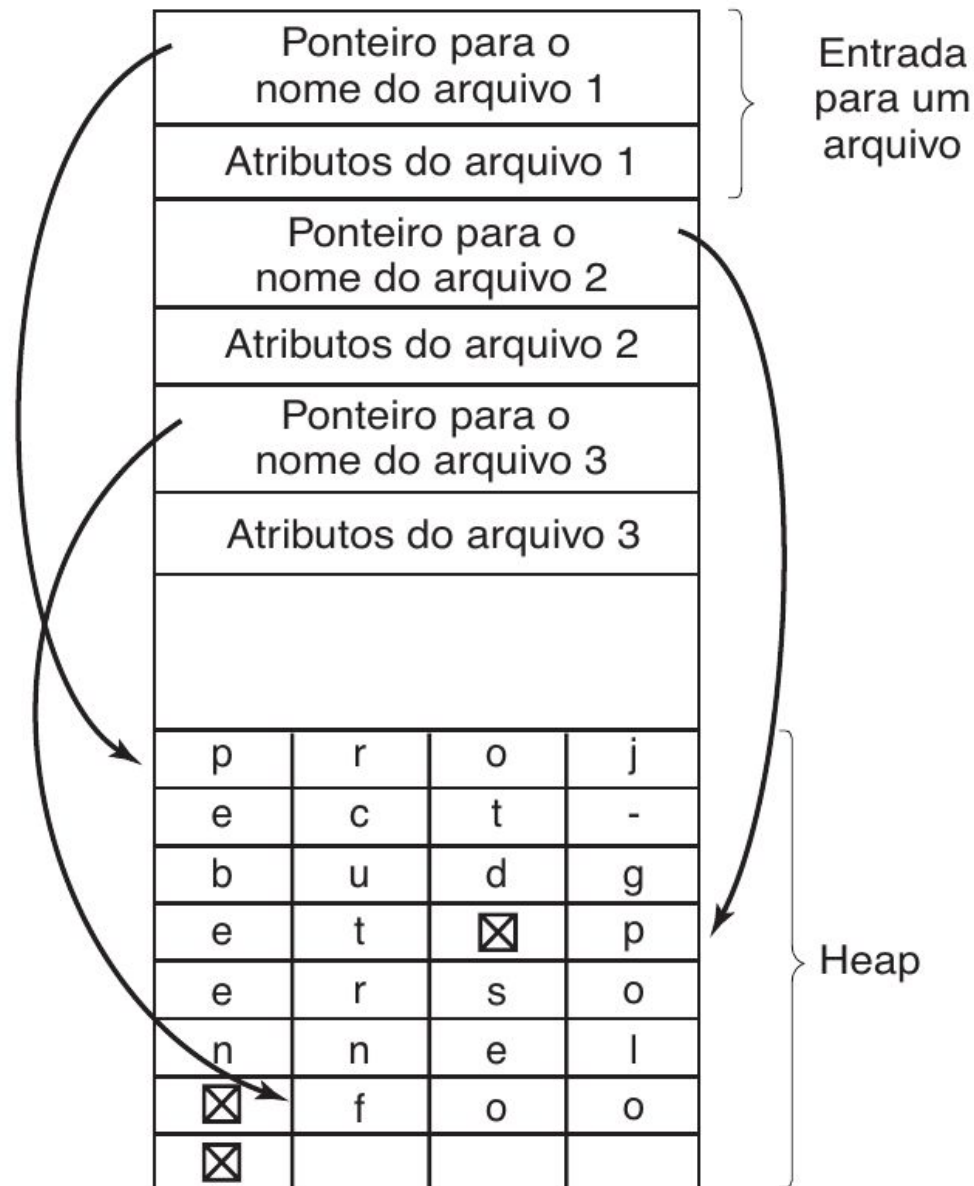
Implementação de Diretórios

- Um outro detalhe da implementação, diz respeito ao tamanho dos nomes dos arquivos (2/2)
 - (a) Uma alternativa é informar inicialmente o tamanho da entrada (nome+atributos) e nome ser o último campo
 - (b) Outra é ter atributos de tamanho fixo e um ponteiro para uma heap de diretórios onde o nome se encontra

Entrada para um arquivo

Tamanho da entrada do arquivo 1			
Atributos do arquivo 1			
p	r	o	j
e	c	t	-
b	u	d	g
e	t	☒	
Tamanho da entrada do arquivo 2			
Atributos do arquivo 2			
p	e	r	s
o	n	n	e
l	☒		
Tamanho da entrada do arquivo 3			
Atributos do arquivo 3			
f	o	o	☒
⋮			

(a)



(b)

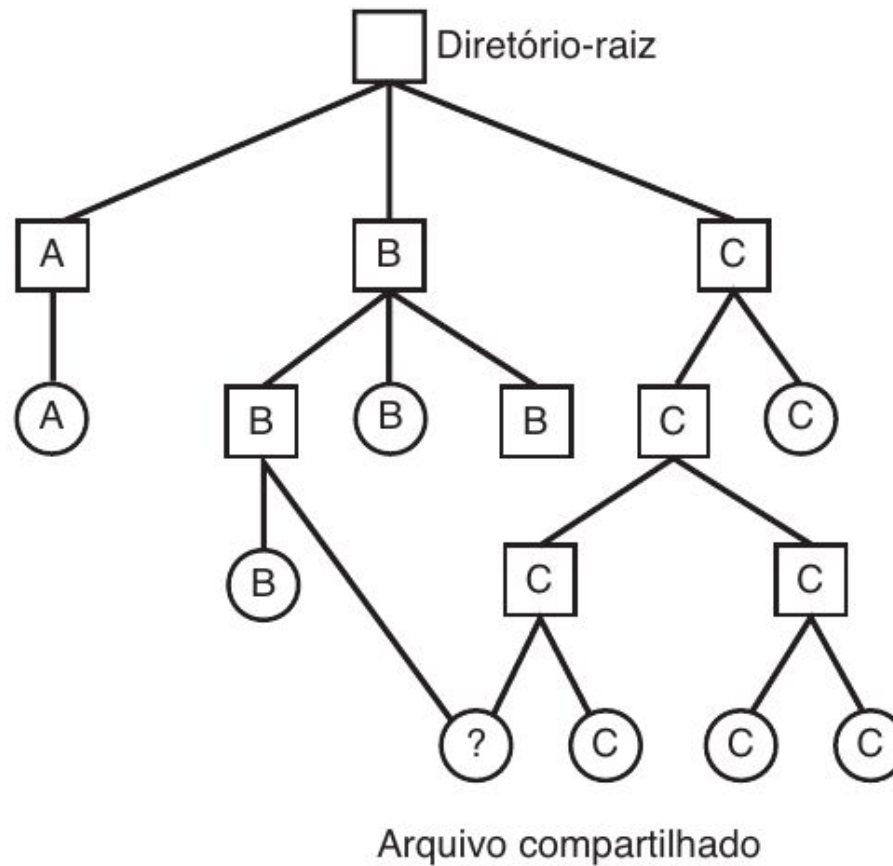
Implementação de Diretórios - Problema da Busca

- No que foi apresentado até agora, busca é feito de forma linear
 - Lento para diretórios longos
 - Pode ser melhorado com uma tabela de hash
 - Aumenta complexidade
- Outra forma de busca é usar cache de resultados

Arquivos Compartilhados

- Quando múltiplos usuários estão trabalhando juntos, as vezes é necessário compartilhar arquivos
- Por consequência, é conveniente que um mesmo arquivo apareça em diferentes diretórios simultaneamente

Arquivos Compartilhados

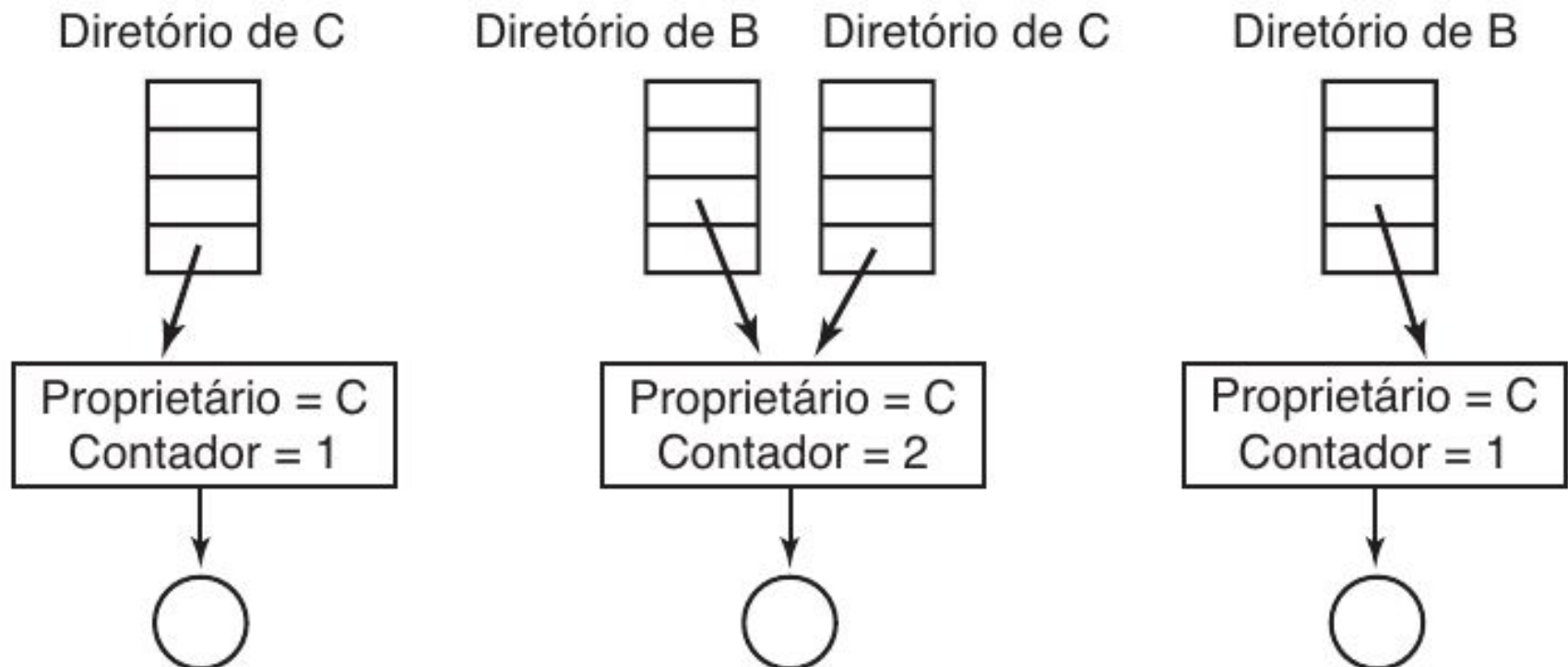


Nesse caso, o sistema de arquivos deixa de ser uma árvore e passa a ser um grafo acíclico orientado (DAG)

Arquivos Compartilhados

- Compartilhar arquivos é conveniente, mas ocasiona problemas
 - Se o compartilhamento for uma cópia, uma modificação não é visível a outros usuários
 - Uma solução é ter a cópia só dos atributos (i-node)
 - Outra solução é utilizar arquivos do tipo link

Arquivos Compartilhados



Questões sobre Sistemas de Arquivos

- Desempenho no acesso
 - Garantir uma busca rápida em um dispositivo lento
- Confiabilidade
 - Perdas
 - Inconsistências

Questões sobre Sistemas de Arquivos

- Para as perguntas anteriores, há soluções em:
 - Software
 - Sistemas de Logs e Sistemas de Jornalameto
 - Hardware
 - Configurações RAID (Redundant Array of Independent Disks)
 - Software e Hardware
 - Sistemas de Backup

Questões sobre Sistemas de Arquivos

- Para as perguntas anteriores, há soluções em:
 - Software
 - Sistemas de Logs e Sistemas de Jornalameto
 - Hardware
 - Configurações RAID (Redundant Array of Independent Disks)
 - Software e Hardware
 - Sistemas de Backup

Uma Solução para o Problema do Desempenho

- Utilizar cache em memória (até funciona mas)
 - Aumenta chance de **perda de dados**
 - Aumenta chance de **inconsistência no sistema**
 - Metadados modificados e conteúdo no disco ainda não
 - Se a escrita for feita de maneira síncrona, fica extremamente lento

Exemplo de Inconsistência

- Remoção de um arquivo:
 - remover a entrada do diretório (1º)
 - liberar o i-node do arquivo (2º)
 - liberar os blocos que pertenciam ao arquivo (3º)
- Quando a operação termina em sucesso, a ordem das operações não importa
- Quando a operação entra em colapso (crash), pode ocorrer o seguinte:
 - Se colapsa em 1º: perde i-nodes e blocos
 - Se colapsa em 2º: se perde os blocos de dados
- Mudar a ordem das operações não resolve o problema:
 - Apenas gera outros problemas

Princípios Básicos de Desempenho e Consistência

- **Preservação:** Dados estáveis no disco não podem ser afetados por uma pane
- **Predição:** Comportamento de recuperação após a pane deve apresentar resultados previsíveis
- **Atomicidade:** Uma operação é completamente realizada ou é considerada como nunca iniciada

Princípios Básicos de Desempenho e Consistência

- Para seguir esses princípios, foram sugeridos os seguintes paradigmas:
 - **Sistemas de Arquivos Estruturados em Logs**
 - **Sistemas de Arquivos Jornalizados**

Sistemas de Arquivos Estruturados em Logs (1/3)

- Agrupa em memórias escritas consideradas pequenas ou randômicas, para serem realizadas de uma única vez de forma sequencial (segmentos)
 - Segmento pode possuir i-nodes, blocos ou bitmaps
- Começo de cada segmento possui um diário (logs) com o que pode ser encontrado nele

Sistemas de Arquivos Estruturados em Logs (2/3)

- I-nodes agora estão dispersos em segmentos
 - Mais difícil de achar
 - Mantido agora um mapa de i-nodes em disco e cache
- Agora, todas as operações de escrita são armazenadas na memória, e periodicamente são escritas no disco para um único segmento

Sistemas de Arquivos Estruturados em Logs (3/3)

- A maior vantagem é o alto desempenho no tempo de escrita
- Como os tipos ficam misturados, necessário garbage collector para organizar os segmentos periodicamente
- Disco cheio causa uma queda brusca no desempenho do disco

Journaling (ou Sistema de Jornalammento)

- Sistemas estruturados em logs são pouco utilizados, mas a solução perante falhas é algo de grande interesse
 - Assim surgiu a ideia de journaling
- Utiliza mecanismos similares aos de bancos de dados
 - Atualizações no sistema são realizadas por meio de transações (o que permite rastreamento)

Journaling (ou Sistema de Jornalammento)

- A ideia é manter um diário registrando o que se vai fazer
 - Antes de o fazer
- Na ocorrência de falha, o diário sabe o que deu errado
 - Consegue concluir o trabalho, ou cancelar sem inconsistência
- NTFS (Microsoft), ext3, ext4 (linux), ReiserFS, APFS (antigo MAC)

Journaling (ou Sistema de Jornalamamento)

- Registro no diário precisam ser **idempotentes**
 - Podem ser repetidas sem prejuízo
- Transações precisam ser **atômicas**
 - Um conjunto de instruções ou operações executam todos em sucesso, ou nada é executado

Journaling (ou Sistema de Jornalammento)

- Uma transação precisa ter a propriedade ACID:
 - **Atomicidade:** ser indivisível da ótica externa
 - **Consistência:** não violar invariantes do sistema
 - **Isolamento:** transações concorrentes não devem interferir uma nas outras
 - **Durabilidade:** efeitos concluídos são permanentes

Journaling (ou Sistema de Jornalammento)

- Princípio de funcionamento:
 - Primitivas básicas: `begin_transaction`,
`end_transaction`, `commit` e `abort`
 - Procedimento de rollback
 - Uma transação não concluída deve manter inalterado o estado do sistema

Journaling (ou Sistema de Jornalameto)

- Modos de journaling (1/3):
 - **Full data**
 - Jornaliza todas as operações sobre o sistema de arquivos e garante a consistência dos dados e dos metadados

Journaling (ou Sistema de Jornalammento)

- Modos de journaling (2/3):
 - **Write behind** (NTFS, ext3, ext4)
 - Jornaliza apenas os metadados
 - As operações de atualização de blocos de dados podem ser perdidas
 - Sistema consistente, mas arquivos podem ter “lixo”

Journaling (ou Sistema de Jornalammento)

- Modos de journaling (3/3):
 - **Ordered Data** (ext3, ext4)
 - Jornaliza apenas os metadados, mas escreve blocos de dados no local definitivo antes de realizar o commit nos metadados e remover a transação
 - Pode acontecer de um bloco novo ter sido escrito no disco, mas ainda não ter sido atualizado como pertencente ao arquivo (perda de dados)

Journaling (ou Sistema de Jornalammento)

- Journaling permite a recuperação rápida de falhas
 - Bom equilíbrio entre desempenho e segurança
- A maior desvantagem é a sobrecarga de escrita
 - Escreve duas vezes, uma no jornal e outra no disco
- Dados recentes podem ser perdidos, mas consistência é garantida

Sistema de Arquivos	Principal SO	Estrutura Principal	Vantagens	Desvantagens	Uso Atual
NTFS	Windows	MFT (Master File Table)	Journaling, arquivos grandes	Complexo	Windows 10/11, Windows Server
ReFS	Windows Server	Evolução do NTFS	Integridade e tolerância a falhas	Compatibilidade limitada	Windows Server, Storage Spaces
ext4	Linux	I-nodes	Estável, rápido, maduro	Sem snapshot	Ubuntu, Debian, Fedora, WSL
XFS	Linux	I-nodes + B+ Trees	Excelente para arquivos grandes	Difícil reduzir partições	RHEL, Oracle Linux
ZFS	Linux/BSD	Copy-on-Write	Integridade, snapshots, RAID nativo	Consome muita RAM	Servidores e NAS
APFS	macOS/iOS	Copy-on-Write	SSDs, snapshots, criptografia	Proprietário	macOS, iPhone, iPad
HFS+	macOS (antigo)	B-Tree	Foi padrão por muitos anos	Substituído pelo APFS	Legado Apple
FAT32	Universal	FAT (File Allocation Table)	Compatibilidade enorme	Arquivos até 4GB	Pendrives antigos
exFAT	Universal	Evolução da FAT	Arquivos grandes	Sem journaling	Pendrives e cartões SD
ISO9660	CDs	Estrutura somente leitura	Simples	Não suporta escrita	CDs
UDF	DVDs/Blu-ray	Evolução do ISO9660	Suporte a gravação	Pouco usado hoje	DVDs e Blu-rays

Conclusão